1

Our laboratory conducts research on computer-generated holography (CGH), using the point cloud generation software U2Dat. Previous studies have implemented refraction and reflection through RayCast and improved visual realism using HDRP. However, several challenges remain: the optical path length has not been implemented, and there are inconsistencies between refracted images rendered in Unity and those generated as point clouds. Moreover, the creation and evaluation of reconstructed images need further development. This study aims to enhance U2Dat's refraction representation, implement optical path length calculations, and generate point clouds and CGHs with photorealistic objects.

2.1

In this study, Unity (version 2022.3.41f1) is used, with the scene template based on HDRP (version 14.0.11). The point cloud generation software “U2Dat,” originally developed in the research by Yuasa et al., is used in a modified form.

U2Dat serves as a preprocessing tool for CGH generation by positioning 3D models and generating point cloud data. By utilizing the Unity rendering engine, U2Dat enables fast and photorealistic point cloud generation. The point cloud data consist of 3D coordinates obtained through RayCast and color information extracted from the color buffer. RayCast is a ray tracing technique in which rays are emitted from a viewpoint (such as a camera) to detect collision points with objects and calculate the distance from those points back to the camera.

2.2

Figure 1 shows the arrangement of objects used in the scene for this study. A single glass sphere is placed in the scene, with its diameter set to 4.0 and its refractive index set to $n = 1.5$. In the figure, **L0** represents the distance from the camera to the surface of the glass sphere, **L1** indicates the optical path length within the sphere, and **L2** denotes the distance from the sphere to the background panel.

2.3

In previous studies, a refraction function using ray tracing was added to U2Dat. A glass object was created using a Material and placed between the camera and the background board. It was confirmed that, under this condition, the refracted image observed in Unity matched the refracted point cloud generated by U2Dat.

However, when the glass object was placed in other positions, a discrepancy arose between the refracted image shown in Unity’s Game view and the ray hit position visible in the Scene view, which represents the generation point of the point cloud. Since the refracted image in the point cloud is determined by this ray hit position, any mismatch with Unity’s refracted image indicates an inaccurate refraction representation.

To address this issue, this study created a new glass material using Shader Graph, as shown in Figure 2, which allows more flexible control of parameters. The processing flow in Shader Graph is as follows: first, the incident vector and the normal vector of the glass object are obtained. These vectors, along with the refractive indices of air and glass, are input into the Refract node to calculate the refracted vector. Then, the value obtained by dividing Sheet Thickness by the refracted vector is calculated and returned to the camera's color buffer.

The Refract node requires four input parameters: the incident vector, the normal vector, the refractive index of air, and the refractive index of the refractive medium (glass). The Sheet Thickness must be manually adjusted, and changing this value affects the appearance of the refracted image. By fine-tuning this parameter, the study aims to match the refracted image in Unity with the refracted image in the point cloud.

2.4

The optical path length is defined as the product of the distance light travels through a medium and the refractive index of that medium. It indicates how far the light would have traveled in a vacuum to achieve the same phase shift and is used in calculations involving phase and interference of light. By incorporating this value, more accurate optical simulations become possible. In this study, we modified the point cloud generation process as follows to reflect the optical path length.

2.5

In this study, to evaluate the newly implemented features in U2Dat, we conducted a simulation of a computer-generated rainbow hologram (CGRH) using a simplified scene. A CGRH produces a visible reconstructed image when illuminated with white light and exhibits a rainbow-like visual effect when the film is moved vertically. Based on the point cloud data generated by U2Dat, interference fringes were calculated and output using a fringe printer to create the CGRH.

3.1

We emitted rays from the camera and examined the positional relationship between the refracted image rendered in Unity and the one generated from the point cloud. Figure 3 shows the results for the glass sphere created using a Material and that created using Shader Graph, while Table 1 presents the coordinates where the rays reached in each case. As seen in Figure 3, the Shader Graph version results in ray endpoints that are closer to the object's contour. The numerical values in Table 1 also confirm that the Shader Graph yields positions closer to the ideal.

3.2

Figure 4 shows the point light source data of the refracted image generated with optical path length taken into account. When calculating nL1 in Equation (1) under the conditions of this

study, the result is 6.0. Next, a ray was cast toward point ①, and the distances between coordinates shown in Figure 1 were measured. The distance within the glass sphere was 4.00005, and the distance from the backboard to the refracted image was 2.0, giving a total of 6.00005. This closely matches the calculated value in the script, confirming that the optical path length was correctly implemented. Similar results were obtained at points ② and ③.

3.3

CGRHs were created using five and 101 parallax images captured along the X-axis. The parameters used for the CGRHs are shown in Table 2. The resulting CGRHs were photographed from three viewpoints—left, front, and right—and the results are presented in Figures 6 and 7, respectively.

As shown in Figure 6, when the number of parallax images is small, the change between viewpoints appears discontinuous, and the motion of the reconstructed image is not smooth. In some viewpoints, refracted images appear doubled, leading to visual artifacts that differ from the actual object. On the other hand, as shown in Figure 7, with a larger number of parallax images, the transitions of the glass sphere and refracted image during viewpoint changes become much smoother, producing a visual effect closer to that of viewing a real object.

4

In this study, we successfully aligned the refracted images rendered in Unity with those generated from the point cloud. We also succeeded in incorporating optical path length into the depth information during point cloud generation. Furthermore, the improved and newly added features were evaluated through the creation of CGRHs.

On the other hand, one remaining issue is that the *Sheet Thickness* parameter used in Shader Graph is not intuitive and can be difficult to understand.

In the future, we plan to implement recursive mirror image processing and photon mapping, and further advance the creation and evaluation of holograms.

ChatGPT 4o と Google AI Studio を比較した。専門用語が見たことある単語であったり文中の英語の略し方が正しい方が ChatGPT 4o のため、今回はそちらを採用した。
元の日本語の文章の簡単化を行い、その文章の英訳をそれぞれの AI に依頼した。